Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build test eligible voting and duplicate controls for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enrol defines eligibility; vote bounds option and flips voted before incrementing count. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Chair account0 enrols account1. Account2 vote must fail. Account1 vote option2 must fail (only0/1). Account1 vote1 succeeds, votes(1)=1 and voted(account1)=true. Repeat vote1 fails.

Expected assertions: votes(1)=1; votes(0)=0; ineligible, invalid-option and duplicate cases rejected.

Extension prompt

Add voting deadline and test votes fail after tester time travel beyond it. Return the exact changed source lines and positive/negative tests.